RAG Chatbot

Build Roadmap & Next Steps

From Notebook Pipeline to Production Chatbot

Stack:	Python 3.13, LangChain, ChromaDB, SentenceTransformers
LLM:	Gemini (Google Generative AI)
UI:	Streamlit
Embeddings:	all-MiniLM-L6-v2
Status:	Retrieval working — chatbot incomplete

Overall Completion

~80% done (single-turn RAG) **~20% remaining (chatbot layer)**

Contents

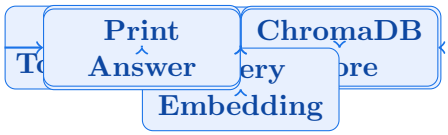
1. What You Have Built So Far

[Done] Completed Components

The following parts of the pipeline are already implemented and verified working in `pipeline.ipynb`:

- **Document loading** — PDFs via `PyPDFLoader`, plain text via `TextLoader`
- **Text chunking** — `RecursiveCharacterTextSplitter` (chunk=1000, overlap=200)
- **Embedding generation** — `all-MiniLM-L6-v2` via SentenceTransformers, normalized vectors
- **Vector store** — ChromaDB persistent collection with UUID-keyed documents
- **Retrieval** — cosine-distance query with configurable top-k and threshold
- **Answer generation** — Gemini LLM via LangChain, single-turn prompting

1.1 Current Architecture (Single-Turn)



[!] Current Limitation: This is a *single-turn* pipeline. Each question is answered independently — there is no memory of previous questions, no file upload at runtime, and no user interface. These are the three things you must build next.

2. What You Need to Build Next

The remaining work has three clearly defined phases:

#	Phase	What it adds	Effort
1	Refactor to modules	Clean, importable `.py` files	Low
2	Conversation memory	Multi-turn chatbot with follow-up support	Medium
3	Streamlit UI	File upload, chat interface, session management	Medium

3. Phase 1 — Refactor Notebook into Modules

[Target] Target

Goal: Extract your notebook code into importable Python files so the Streamlit app can use it cleanly without re-running notebook cells.

3.1 Recommended Project Structure

```
your_project/
|-- data_files/
|    |-- pdf_files/
|    `-- text_files/
|-- chroma_db/              # persisted vector store
|-- pipeline/
|    |-- __init__.py
|    |-- loader.py          # load_pdf_documents,
   load_text_documents
|    |-- chunker.py         # split_documents
|    |-- embedder.py        # EmbeddingManager2
|    |-- vector_store.py    # VectorStoreManager
|    |-- retriever.py       # RAGretriever
|    `-- generator.py       # generate_answer (with memory)
|-- app.py                  # Streamlit entry point
|-- .env                    # GOOGLE_API_KEY (never commit!)
`-- requirements.txt
```

3.2 What Goes in Each File

`pipeline/loader.py` — move both `load_pdf_documents` and `load_text_documents` functions here, unchanged.

`pipeline/chunker.py` — move `split_documents` function here.

`pipeline/embedder.py` — move `EmbeddingManager2` class here.

`pipeline/vector_store.py` — move `VectorStoreManager` class here. Add a method:

```python
def clear_and_reset(self):
    """Delete and recreate the collection to avoid stale data.
    """
    try:
        self.client.delete_collection(self.collection_name)
    except Exception:
        pass
    self._initialize_vector_store()
```

`pipeline/retriever.py` — move `RAGretriever` class here. Increase default `threshold_dist` to `1.5`.

`pipeline/generator.py` — this is *new* (see Phase 2).

3.3 Target: Working Import Test

After refactoring, you should be able to run this from the project root without errors:

```python
from pipeline.loader import load_pdf_documents
from pipeline.embedder import EmbeddingManager2
from pipeline.vector_store import VectorStoreManager
from pipeline.retriever import RAGretriever
```

```
print("All imports OK")
```

4. Phase 2 — Add Conversation Memory

[Target] Target

Goal: Make the chatbot remember previous turns so users can ask follow-up questions like *"What about his GPA?"* after asking about education.

4.1 Why Memory is Needed

Your current LLM call is:

```
prompt = f"Answer this question based on context.\n\nQuestion: {query}\n\nContext:\n{context}"
response = llm.invoke(prompt)
```

Every call is fresh. The LLM has no idea what was said before. This breaks any follow-up question that uses pronouns or references like *"his"*, *"that project"*, *"explain more"*.

4.2 Solution: Pass Chat History into the Prompt

Create `pipeline/generator.py`:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from typing import List, Dict
import os
from dotenv import load_dotenv

load_dotenv()

def build_llm(model: str = "gemini-1.5-flash", temperature: float = 0):
    """Initializes the LLM. Call once and reuse."""
    return ChatGoogleGenerativeAI(model=model, temperature=temperature)


def generate_answer(
    query: str,
    retrieved_docs: List[Dict],
    chat_history: List[Dict],
    llm
) -> str:
    """
    Generates an answer using retrieved context and prior chat history.

    Args:
        query:            The current user question.
```

```
        retrieved_docs: List of dicts from RAGretriever.retrieve().
        chat_history:   List of {'role': 'user'/'assistant', 'content': '...'}.
        llm:            LangChain LLM instance.

    Returns:
        Answer string from the LLM.
    """
    # Build context string from retrieved chunks
    if retrieved_docs:
        context = "\n\n".join(
            f"[Source: {doc['metadata'].get('filename', 'unknown')}]\n{doc['document']}"
            for doc in retrieved_docs
        )
    else:
        context = "No relevant documents found."

    # Keep only the last 6 messages (3 turns) to stay within token limits
    recent_history = chat_history[-6:]
    history_text = "\n".join(
        f"{msg['role'].capitalize()}: {msg['content']}"
        for msg in recent_history
    )

    prompt = f"""You are a helpful assistant that answers questions based on the provided documents.
Use the context below to answer the user's question accurately.
If the answer is not in the context, say so clearly instead of making something up.

--- CHAT HISTORY ---
{history_text}

--- DOCUMENT CONTEXT ---
{context}

--- CURRENT QUESTION ---
{query}

Answer:"""

    response = llm.invoke(prompt)

    if isinstance(response.content, list):
        return "\n".join(
            part.get("text", "") for part in response.content
            if isinstance(part, dict)
        ).strip()
```

```
    return response.content.strip()
```

4.3 How Chat History Should Be Managed

In Streamlit, store the history in `st.session_state`:

```
# Initialize once
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []

# After each Q&A turn:
st.session_state.chat_history.append({"role": "user",      "content": query})
st.session_state.chat_history.append({"role": "assistant", "content": answer})
```

Token Budget: Keep only the last 6 messages (3 turns) in the prompt. Passing the entire history will eventually exceed the LLM context window and raise an error.

5. Phase 3 — Streamlit Dashboard

[Target] Target

Goal: Build a browser-based chat interface where users can upload new files and have a multi-turn conversation with their documents.

5.1 Required UI Components

Component	Purpose
Sidebar — file uploader	User uploads PDF or TXT; triggers ingestion pipeline
Sidebar — indexed files	Shows which files are already in the vector store
Chat window	Displays the full conversation (user + assistant turns)
Input box	User types question; Enter submits
Clear chat button	Resets `chat_history` in session state

5.2 Full `app.py` Scaffold

```
import streamlit as st
import tempfile
import os
from pipeline.loader import load_pdf_documents, load_text_documents
from pipeline.chunker import split_documents
from pipeline.embedder import EmbeddingManager2
```

```python
from pipeline.vector_store import VectorStoreManager
from pipeline.retriever import RAGretriever
from pipeline.generator import build_llm, generate_answer

#            Page config


st.set_page_config(page_title="RAG Chatbot", page_icon="    ", layout="wide")
st.title("     RAG Document Chatbot")

#            Session state initialization


@st.cache_resource
def load_pipeline():
    """Load heavy components once and cache across reruns."""
    embedding_manager = EmbeddingManager2()
    vector_store = VectorStoreManager(collection_name="chatbot_docs")
    retriever    = RAGretriever(vector_store, embedding_manager, threshold_dist=1.5)
    llm          = build_llm()
    return embedding_manager, vector_store, retriever, llm

embedding_manager, vector_store, retriever, llm = load_pipeline()

if "chat_history"    not in st.session_state:
    st.session_state.chat_history    = []
if "indexed_files"   not in st.session_state:
    st.session_state.indexed_files   = []

#            Sidebar     file upload


with st.sidebar:
    st.header("     Upload Documents")
    uploaded_file = st.file_uploader(
        "Upload a PDF or TXT file",
        type=["pdf", "txt"]
    )

    if uploaded_file and uploaded_file.name not in st.session_state.indexed_files:
        with st.spinner(f"Indexing {uploaded_file.name}..."):
            # Save to temp file
            suffix = ".pdf" if uploaded_file.type == "application/pdf" else ".txt"
            with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
```

```
                tmp.write(uploaded_file.getvalue())
                tmp_path = tmp.name

            # Load, chunk, embed, store
            if suffix == ".pdf":
                docs = load_pdf_documents(os.path.dirname(tmp_path))
            else:
                docs = load_text_documents(os.path.dirname(tmp_path))

            chunks     = split_documents(docs)
            embeddings = embedding_manager.generate_embeddings(chunks)[0]
            vector_store.add_docs(chunks, embeddings)

            st.session_state.indexed_files.append(uploaded_file.name)
            os.unlink(tmp_path)
        st.success(f"    Indexed: {uploaded_file.name}")

    st.divider()
    st.subheader("     Indexed Files")
    if st.session_state.indexed_files:
        for fname in st.session_state.indexed_files:
            st.write(f"    {fname}")
    else:
        st.caption("No files indexed yet.")

    if st.button("        Clear Chat"):
        st.session_state.chat_history = []
        st.rerun()

#           Main     chat display


for msg in st.session_state.chat_history:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

#           Input


if query := st.chat_input("Ask something about your documents ..."):
    # Show user message
    with st.chat_message("user"):
        st.markdown(query)
    st.session_state.chat_history.append({"role": "user", "content": query})

```

```
    # Retrieve + generate
    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            retrieved = retriever.retrieve(query, top_k=5)
            answer    = generate_answer(
                query, retrieved,
                st.session_state.chat_history,
                llm
            )
        st.markdown(answer)

    st.session_state.chat_history.append({"role": "assistant",
       "content": answer})
```

5.3 Running the App

```
# Install any missing dependency
pip install streamlit python-dotenv

# Run from the project root
streamlit run app.py
```

6. Known Bugs to Fix Before Building the UI

These were identified during review of `pipeline.ipynb` and must be resolved first:

#	Bug	Impact	Fix
1	`threshold_dist=1.0` too low	Empty retrieval, no answer printed	Change to `1.5`
2	Duplicate documents in ChromaDB	Stale data accumulates each run	Add `clear_and_reset()` before ingestion
3	API key hardcoded in notebook	Security risk, key exposed	Use `.env` + `python-dotenv`
4	`gemini-3-flash-preview` model name	May break on API validation	Use `gemini-1.5-flash`

7. Optional Improvements (After Core is Working)

Once the chatbot is running end-to-end, consider these upgrades roughly in order of impact:

7.1 Source Attribution

Show which file and page each answer came from. This builds user trust and is easy to add since metadata is already stored in ChromaDB:

```
for doc in retrieved_docs:
    fname = doc['metadata'].get('filename', 'unknown')
    page  = doc['metadata'].get('page', '?')
    st.caption(f"Source: {fname}, page {page}")
```

7.2 Hybrid Search (BM25 + Vector)

Your current retrieval is pure dense vector search. Adding BM25 keyword search alongside it (and merging results with Reciprocal Rank Fusion) significantly improves recall for exact terms like names, codes, and technical keywords.

```
from rank_bm25 import BM25Okapi

# Index all chunk texts
tokenized = [doc.page_content.split() for doc in chunks]
bm25 = BM25Okapi(tokenized)

# Retrieve with BM25
scores = bm25.get_scores(query.split())
top_bm25_indices = scores.argsort()[-5:][::-1]
```

7.3 Smarter Chunking with Metadata Filtering

Add file-level filters to retrieval so users can ask *"only search the resume"* without results from unrelated documents:

```
results = self.collection.query(
    query_embeddings=[query_embedding.tolist()],
    n_results=top_k,
    where={"filename": {"$eq": "Resume.pdf"}}  # metadata
        filter
)
```

7.4 Streaming Responses

Stream the LLM output word-by-word in Streamlit for a ChatGPT-like feel:

```
with st.chat_message("assistant"):
    placeholder = st.empty()
    full_response = ""
    for chunk in llm.stream(prompt):
        full_response += chunk.content
        placeholder.markdown(full_response + "   ")
    placeholder.markdown(full_response)
```

8. Summary Checklist

Task	File(s)	Priority
Fix `threshold_dist` to 1.5	`retriever.py`	Critical
Move API key to `.env`	`.env`	Critical
Refactor notebook into `pipeline/` modules	5 new `.py` files	High
Add `clear_and_reset()` to VectorStoreManager	`vector_store.py`	High
Write `generator.py` with memory-aware prompt	`generator.py`	High
Build `app.py` Streamlit interface	`app.py`	High
Add source attribution in UI	`app.py`	Medium
Add streaming response	`app.py`	Medium
Add BM25 hybrid search	`retriever.py`	Optional
Add metadata filters for per-file queries	`retriever.py`	Optional

Recommended build order:

Fix bugs → Refactor to modules → Add memory in `generator.py` → Build Streamlit UI → Test end-to-end → Add streaming + source attribution